Deep Learning-Based Face Mask Detection

Yuchen Ding
Department of MIE
University of Toronto
yuchen.ding@mail.utoronto.ca

Yu Tao
Department of MIE
University of Toronto
francis.tao@mail.utoronto.ca

Jiayi Liu
Department of MIE
University of Toronto
ljiayi.liu@mail.utoronto.ca

Abstract

This project explores the development of a deep learning-based system for face mask detection to address public health concerns during the COVID-19 pandemic. Utilizing pre-trained object detection models, Faster R-CNN and YOLOv5, the system classifies individuals into three categories: with mask, without mask, and mask worn incorrectly. The dataset, comprising 853 labeled images, underwent extensive preprocessing and augmentation to enhance model generalization. Both models were trained and evaluated using standard metrics such as precision, recall, and mean Average Precision (mAP). YOLOv5 demonstrated superior real-time performance and robustness on small objects, while Faster R-CNN achieved acceptable accuracy in complex scenarios but suffered from overfitting and slower inference times. The results highlight YOLOv5 as the preferred model for real-time mask detection applications, with future improvements targeting dataset balance, sophisticated model versions, and enhanced architecture.

Assentation of Teamwork
Yuchen Ding: Project proposal & briefing; YOLO code revision; Poster design; Final Report writing
Yu Tao: Faster R-CNN implementation code; Poster revision; Final Report revision
Jiayi Liu: Yolo implementation code

1 Introduction

The COVID-19 pandemic has presented a crucial demand for face masks as a vital public health measure to curb the spread of the virus. Ensuring that individuals wear masks correctly in public spaces has become a critical concern, especially in high-risk areas like hospitals, airports, and offices. Manual monitoring of mask compliance requires tremendous amounts of labour, resulting in low efficiency, and high labour costs.

This project aims to develop a deep learning-based face mask detection system using two pre-trained object detection models: **Faster R-CNN** and **YOLOv5**. By leveraging advanced object detection techniques, the system seeks to automatically identify whether individuals are wearing masks properly, improperly, or not at all. The comparison between Faster R-CNN and YOLOv5 is conducted to determine the best-performing model in terms of accuracy, speed, and generalization.

2 Preliminaries and Problem Formulation

The COVID-19 pandemic has necessitated the widespread use of face masks as a crucial public health measure to prevent virus transmission. Monitoring mask compliance manually is labour-intensive and impractical on a large scale. Therefore, we aim to develop an automated face mask detection system using deep learning techniques.

The primary objective of this project is to create a robust face mask detection system capable of classifying faces into three categories:

1. Mask Worn Properly: The mask covers both the nose and mouth adequately.

2. No Mask: The individual is not wearing a mask.

3. Mask Worn Improperly: The mask is worn but not correctly positioned (e.g., below the nose, on the chin).

To tackle this problem, we needed to understand several new concepts and techniques:

- Deep Learning and Convolutional Neural Networks (CNNs): Essential for feature extraction and pattern recognition in images.

- Object Detection Models:
  - **Faster R-CNN**: Combines region proposal with classification, suitable for detecting multiple objects within images (Ren et al., 2015).

  - **YOLOv5**: Performs one-stage object detection with high speed and accuracy, making it ideal for real-time applications (Glenn Jocher, 2020).

- Yolov5: Predict bounding boxes and class probabilities in one step, supports auto data augmentation, and performs better on datasets with many small objects due to its grid-based detection approach.

- Data Preprocessing and Augmentation: Techniques to prepare and enhance the dataset for better model performance.

- Hyperparameter Tuning: Adjusting model parameters such as learning rate and batch size to optimize performance and prevent overfitting.

- Evaluation Metrics: Understanding metrics like precision, recall, F1-score, and Mean Average Precision (mAP) to assess the model's accuracy.

3 Solutions via Deep Learning

Our solution involves utilizing deep learning techniques to develop a face mask detection system using the Faster R-CNN and Yolov5 model and comparing the loss for the two models to pick the best one. The key components of our approach are the dataset, the model architecture, the loss functions, and the training and testing methodologies.

3.1 Dataset

We are using the **Kaggle Face Mask Detection** dataset, which contains 853 labelled images with bounding boxes and class labels corresponding to the three categories of mask usage. The dataset offers diversity in terms of lighting conditions, angles, backgrounds, and mask types, which is essential for training a robust model (Kaggle, 2020).

3.2 Model Architecture:

Faster R-CNN - ResNet50

The Faster Region-Based Convolutional Neural Network (Faster R-CNN) is selected for its efficiency and accuracy in object detection tasks (Ren et al., 2015). The architecture comprises two main components:

1. Region Proposal Network (RPN): Generates regional proposals where objects might be located in the image.

2. Fast R-CNN Detector: Performs classification and refines bounding boxes on the proposed regions.

This trainable model is capable of real-time object detection and is well-suited for detecting multiple objects within an image, which is critical for our application (Ren et al., 2015).

YOLOv5

YOLOv5 is a one-stage object detection model designed for real-time applications. It directly predicts bounding boxes and class probabilities in a single forward pass (Glenn Jocher, 2020). Its architecture includes:

- **Backbone**: CSPDarknet53 for feature extraction.
- **Neck**: Path Aggregation Network (PANet) to combine features across scales.
- **Head**: Anchor-based prediction for bounding boxes and class probabilities.

YOLOv5 is optimized for speed and performs well on small objects like improperly worn masks. It also supports advanced data augmentation techniques, such as mosaic augmentation, which enhance the diversity of training data (Glenn Jocher, 2020).

3.3 Loss Functions

The project leverages the Faster R-CNN architecture, which inherently uses **Multi-task Loss** as part of its design:

- **Bounding Box Regression Loss**: A smooth L1 loss for predicting accurate bounding box coordinates.
- **Classification Loss**: Typically, a cross-entropy loss for class prediction ("with mask", "without mask", "mask worn incorrectly")

By optimizing both classification and bounding box regression loss simultaneously, the model could learn accurately identify and locate

the faces with or without masks.

YOLOv5 Loss Functions:

YOLOv5 uses a composite loss function:

- **CIoU Loss**: For bounding box regression, emphasizing overlap, distance, and aspect ratio consistency.
- **Objectness Loss**: Binary cross-entropy loss for predicting whether a bounding box contains an object.
- **Classification Loss**: Binary cross-entropy for classifying the detected objects.

3.4 Data Preprocessing

Data preprocessing steps include:

- Resizing and Normalization: All images are resized to a uniform dimension, and pixel values are normalized to facilitate model convergence.
- Data Augmentation: Techniques such as blurring, rotation, flipping, scaling, random brightness, and colour jittering are applied to increase dataset variability and improve model generalization.
- Handling Class Imbalance: If any class is underrepresented, methods like oversampling the minority class or using class weights in the loss function are employed.

3.5 Hyperparameter Tuning

Key hyperparameters are fine-tuned to optimize model performance:

- Optimizer: Experimentation with optimizers like Stochastic Gradient Descent (SGD) with momentum and Adam optimizer.
- Learning Rate: Starting with an initial learning rate (e.g., 0.01) and adjusting it based on validation performance.
- Batch Size and Epochs: Determined based on hardware limitations, with training conducted over multiple epochs (e.g., 10–50 epochs) to ensure convergence.
- Regularization: Techniques such as weight decay and dropout are used to prevent overfitting.

3.6 Testing Methodology

After training, the model is evaluated on a separate test set to assess its generalization capabilities.

- Precision, Recall, and F1-Score: To measure the accuracy of the model's predictions.
- Mean Average Precision (mAP): To evaluate the model's performance across different thresholds.
- Intersection over Union (IoU): To measure the overlap ratio between prediction and ground truth bounding boxes.
- Confusion Matrix: To analyze the types of misclassifications and areas for improvement.
- Non-Maximum Suppression (NMS): To drop the duplicate detection based on different levels of threshold.
- Visualization of bounding boxes and predicted labels to verify model performance qualitatively.

4 Implementation

4.1 Library and Tools

The implementation requires the application of several libraries to facilitate the training of the face mask detection system. PyTorch (torchvision) is used for implementing the Faster R-CNN model, providing flexibility and support for GPU acceleration via CUDA. The Ultralytics YOLO framework simplifies the training and evaluation of YOLOv5, including its advanced features like mosaic augmentation and anchor box optimization. Data augmentation is performed using the Albumentations library, which supports a wide range of transformations to enhance dataset variability. Visualization of results, including bounding boxes and predictions, is facilitated by matplotlib. All computations are accelerated using NVIDIA CUDA GPUs, enabling efficient training and inference for both models.

4.2 Design and Key Components

The implementation of the face mask detection project involves several stages, from data preprocessing to evaluation. The data preprocessing stage standardizes the input images for consistency across both models. All images are resized to (376, 376) and normalized to pixel values in the range [0, 1] for the sake of computational cost. Data augmentation techniques such as flipping, scaling, and rotation are applied to enhance dataset variability, with a focus on addressing minority classes like "mask worn incorrectly." Annotations are processed differently for each model: Faster R-CNN uses the Pascal VOC XML format, while YOLOv5 annotations

are converted to their required format.

Two distinct object detection models are deployed: **Faster R-CNN** and **YOLOv5**. Faster R-CNN is a two-stage detection model that generates regional proposals using a Region Proposal Network (RPN) before classifying and refining bounding boxes. It is well-suited for high-accuracy detection tasks and includes a classification head for predicting mask categories and a regression head for bounding box refinement.

In contrast, YOLOv5 is a one-stage detection model designed for real-time applications. It directly predicts bounding boxes, objectness scores, and class probabilities in a single forward pass. YOLOv5's architecture incorporates a CSPDarknet53 backbone for feature extraction, a Path Aggregation Network (PANet) for feature fusion, and a detection head optimized for efficient inference. YOLOv5 also supports advanced augmentation techniques such as mosaic augmentation, which are particularly effective for small datasets.

4.3 Training Pipeline

Faster R-CNN

The Faster R-CNN model was trained using a carefully structured pipeline designed to maximize accuracy and minimize overfitting. The images in the dataset were preprocessed by resizing them to a uniform dimension of (376, 376) and normalizing their pixel values to the range [0, 1]. The annotations, originally in Pascal VOC XML format, were parsed and prepared to meet the input requirements of the Faster R-CNN architecture.

The training process involved optimizing the model parameters using Stochastic Gradient Descent (SGD) with momentum and weight decay. To adjust the learning rate dynamically, a ReduceLROnPlateau scheduler was employed. Mixed precision training was utilized with GradScaler and autocast, which allowed for efficient use of GPU memory and faster computation without compromising numerical accuracy.

The total loss for the model was calculated as the sum of classification loss, using cross-entropy for class predictions, and bounding box regression loss, implemented with Smooth L1 loss. During the training process, the model's performance was assessed using the mean Average Precision (mAP) metric. To prevent overfitting or stagnation, the training procedure included a reset mechanism to restore the learning rate to its initial value if the validation loss did not improve for 20 consecutive epochs. Training was terminated early if the mAP@0.5 score exceeded the predefined threshold of 0.87.

The training progress was visualized by plotting the loss curves and mAP scores across epochs, enabling a clear understanding of the model's performance trends over time.

YOLOv5 Training Procedure

The YOLOv5 model was trained using the Ultralytics YOLO framework, which provided a streamlined and efficient process for object detection tasks (Glenn Jocher, 2020). The dataset images were resized to (640, 640) during training, and annotations were converted from Pascal VOC XML format to YOLO's required format, which uses normalized bounding box coordinates. Advanced data augmentation techniques, such as mosaic augmentation, flipping, and scaling, were automatically applied by the YOLOv5 framework, enhancing the diversity of the training data and improving the model's robustness.

The YOLOv5 model was initialized with pre-trained weights from yolov5s.pt, which had been trained on the COCO dataset (Glenn Jocher, 2020). The model's parameters were optimized using the Adam optimizer with an initial learning rate that was dynamically adjusted using a ReduceLROnPlateau scheduler. This configuration ensured a gradual reduction in the learning rate if no improvement in validation loss was observed.

The model's performance was evaluated using standard object detection metrics, including Precision, Recall, F1-score, and mAP. The YOLO framework provided detailed logs and visualizations of the training process, including loss curves and validation metrics, making it easier to track progress and identify areas for improvement. During training, the best-performing weights were automatically saved based on validation performance. Training was terminated if further improvements in mAP were not observed, ensuring computational efficiency.

4.4 Evaluation

To assess model performance, both Faster R-CNN and YOLOv5 are evaluated using standard metrics, including precision, recall, F1-score, and mean Average Precision (mAP). Inference speed is also measured to compare the real-time applicability of each model. Faster R-CNN demonstrates strong performance in scenarios requiring high accuracy but is slower during inference. YOLOv5, on the other hand, achieves real-time detection speeds while maintaining competitive accuracy.

Qualitative analysis is conducted by visualizing bounding boxes and predictions on test images, highlighting the strengths and weaknesses of each model. Faster R-CNN excels in complex background scenarios, while YOLOv5 is more effective in detecting objects quickly and handling small objects like improperly worn masks.

4.5 Algorithms

Data augmentation is implemented using the Albumentations library, with transformations such as horizontal flipping, brightness adjustment, and scaling applied to the training dataset. The Faster R-CNN model is implemented using PyTorch, with a ResNet-50 backbone pre-trained t. YOLOv5 is implemented using the Ultralytics framework, leveraging the yolov8s.pt weights as the base model.

Faster R-CNN Training Loop Pseudocode:

```
for epoch in range(num_epochs):
    model.train()
    for images, targets in train_loader:
        optimizer.zero_grad()
        loss_dict = model(images, targets)
        total_loss = sum(loss_dict.values())
        total_loss.backward()
        optimizer.step()
```

YOLOv5 Training Loop Pseudocode:

```
from ultralytics import YOLO

# Load pre-trained YOLOv5 model
model = YOLO('yolov5s.pt')    # Load a pre-trained YOLOv5 model

# Train the model
results = model.train(
        data='/content/data.yaml',
        epochs=70,
        imgsz=640,
        batch=14,
        cos_lr=True,
        project='face_mask_yolo',        # Project directory
        name='experiment1',                  # Experiment name
```

5 Numerical Experiments

To evaluate the effectiveness of our YOLOv5-based design for face mask detection, a series of numerical experiments were conducted using a custom dataset containing three classes: **with_mask**, **without_mask**, and **mask_weared_incorrect**. The experiments focused on training the model, validating its performance, and testing its generalization on unseen data. In addition, key metrics were compared against standard benchmarks reported in the literature for object detection tasks.

YOLOv5

The YOLOv5s model was chosen due to its balance between speed and accuracy. The dataset was divided into training (70%), validation (15%), and test (15%) subsets, maintaining a similar class distribution across all splits. The experiments tried various sets of configurations by tuning batch size, number of epochs and learning rate, resulting in the following best configuration:

- **Epochs**: 70
- **Batch Size**: 14
- **Image Size**: 640×640
- **Learning Rate Scheduler**: Cosine decay
- **Device**: NVIDIA A100 GPU
- **Built-in Augmentations**: YOLOv5's automatic data augmentation features were utilized, including mosaic augmentations, flips, and scaling.

Faster R-CNN

The Faster R-CNN model was known for its high accuracy. The dataset was divided into training (70%), validation (15%), and test (15%) subsets, maintaining a similar class distribution across all splits. Several versions with different techniques were implemented to reduce overfitting.

Plain model with default settings' configurations (pretrained weights):

- **Epochs**: 20
- **Batch Size**: 14
- **Image Size**: 356×356
- **Class Weights**: ['with mask': 1, 'without mask': 4.78, 'mask worn incorrectly': 25.87]
- **Initial Learning Rate**: 0.01

- **Learning Rate Scheduler**: ReduceOnPlateau with a factor of 0.1 and patience of 3
- **Trainable backbone layers**: 3 (default)
- **Device**: NVIDIA A100 GPU
- **Augmentations**: Manual augmentations such as horizontal flip, channel shuffle, random brightness and contrast, and blurring were applied.

Model with frozen layers in backbone:

- **Trainable backbone layers**: 2
- Others Not Changed

Model with dropout applied to dense layer (first 2 fully connected layers in RoI heads):

- **Dropout Rate**: 0.5
- Others Not Changed

Model with dropout + L2 regularization added:

- **Dropout Rate**: 0.5
- **Weight Decay**: 0.01
- Others Not Changed

Figure 1: Faster R-CNN Default Learning Curves

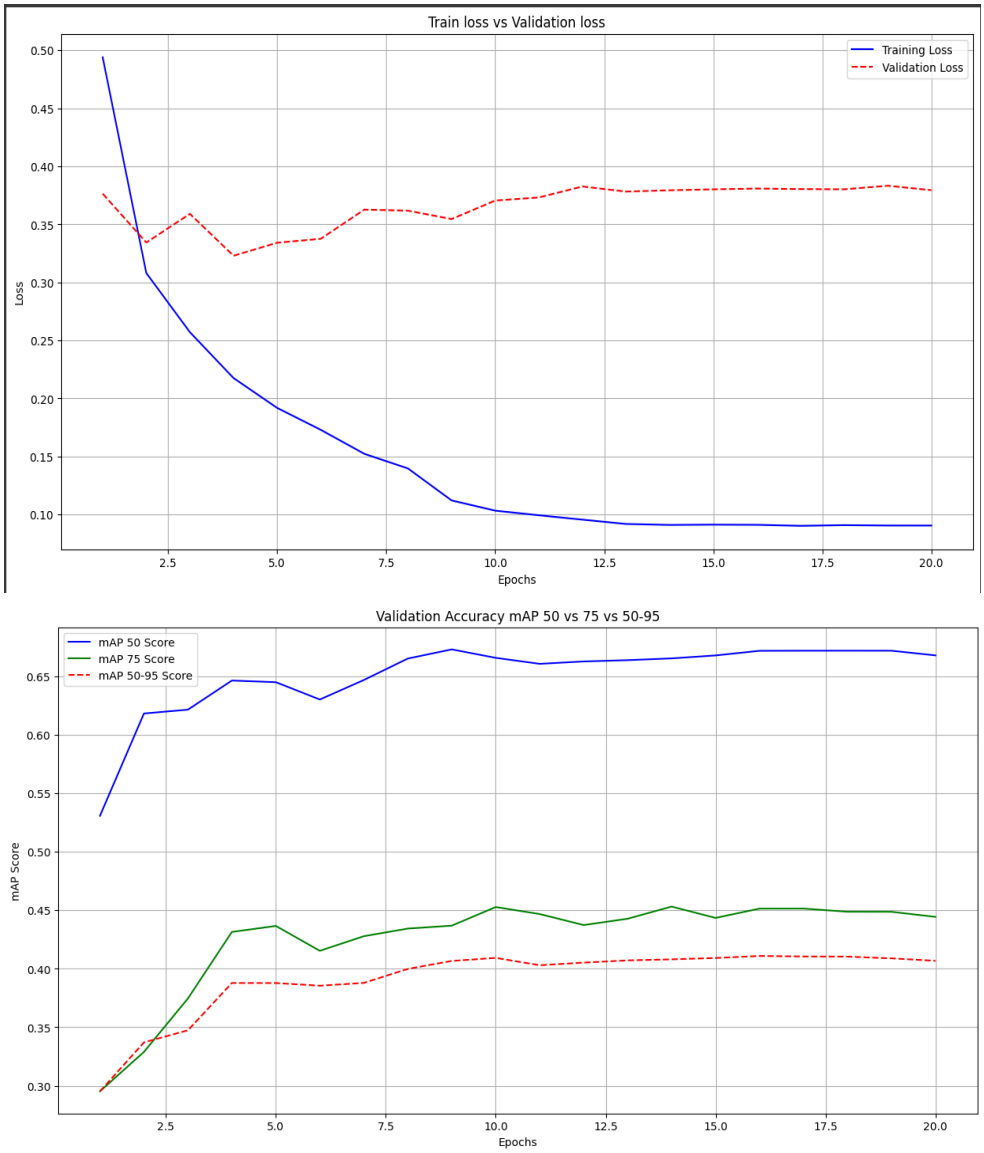


Figure 2: Faster R-CNN Freeze Learning Curves

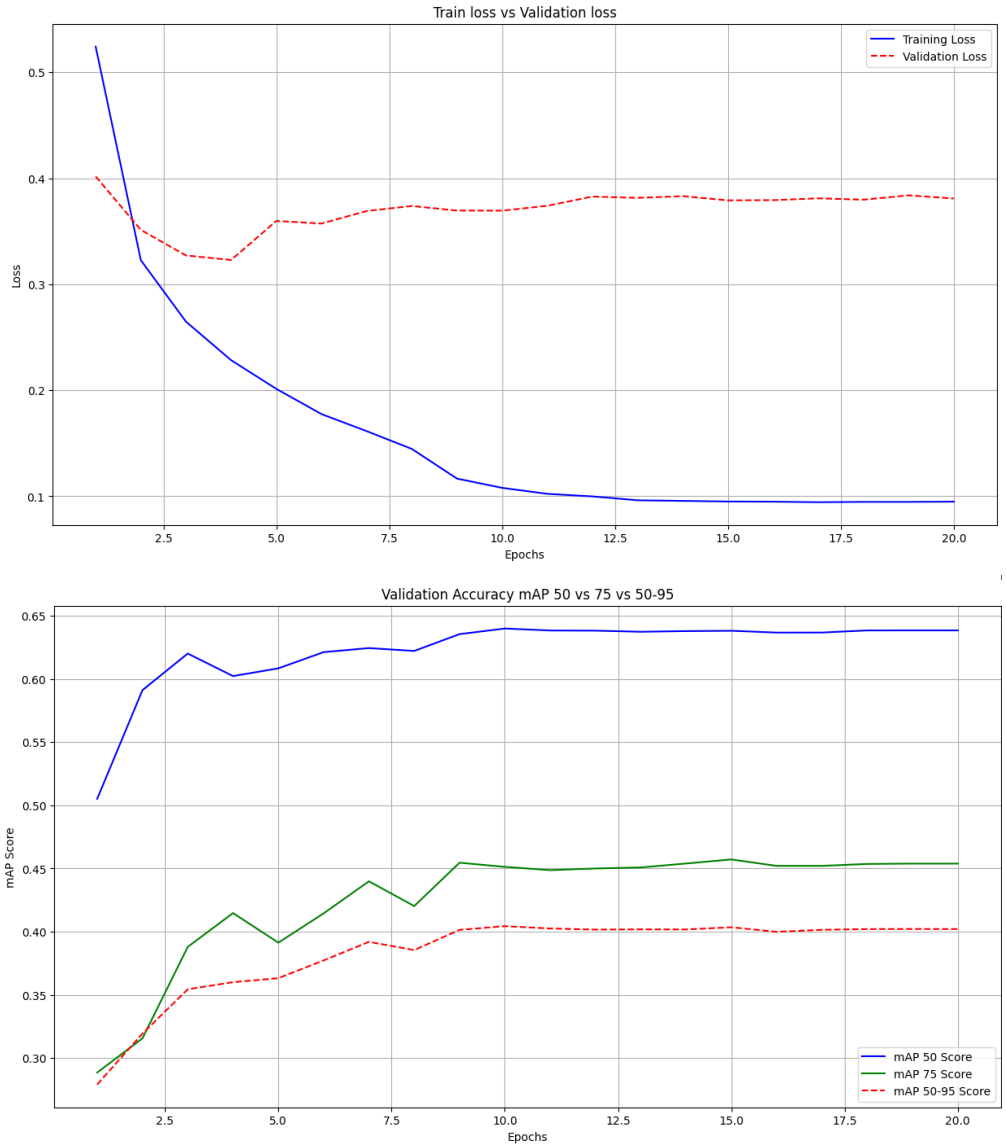


Figure 3: Faster R-CNN Dropout Learning Curves

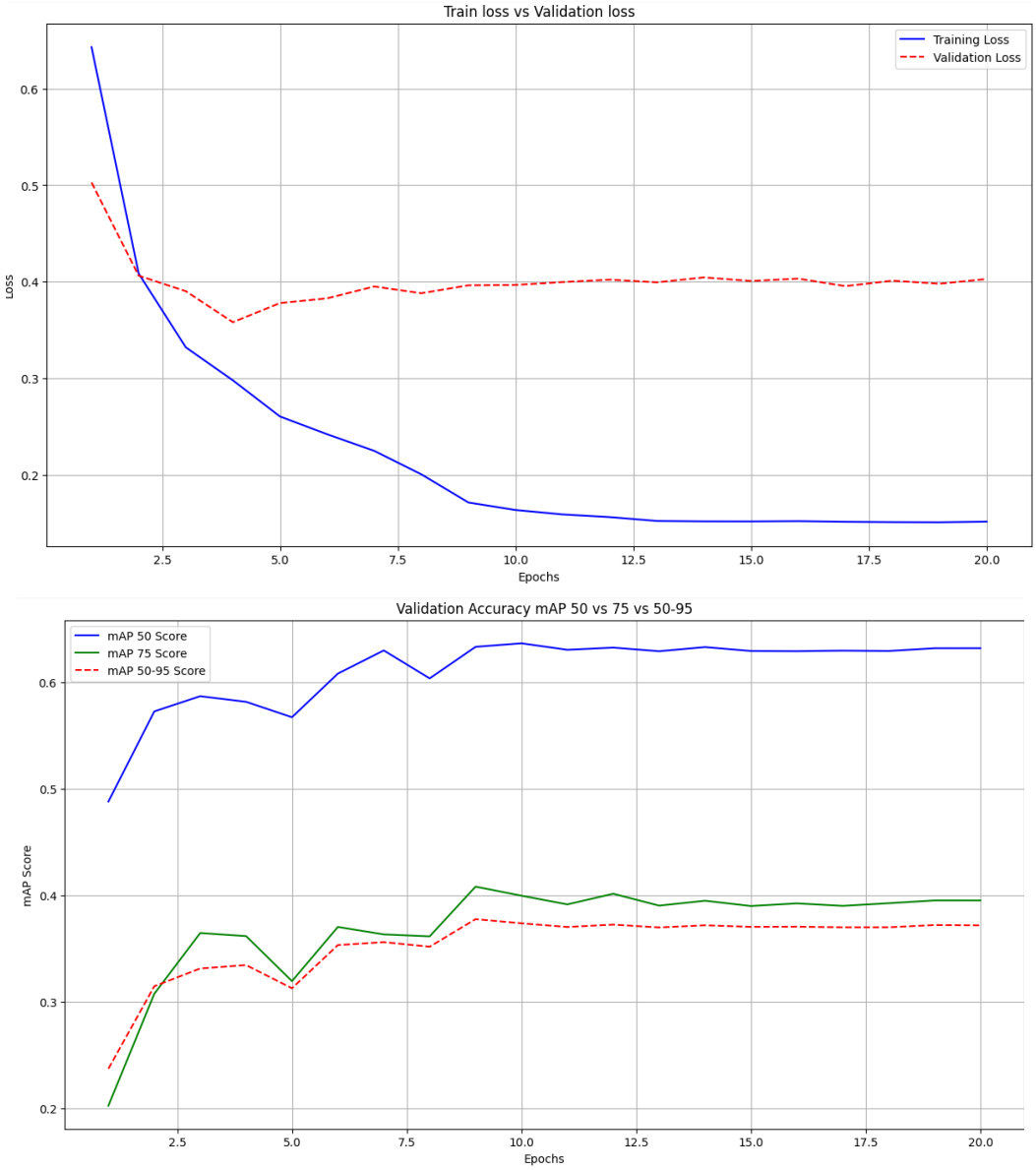
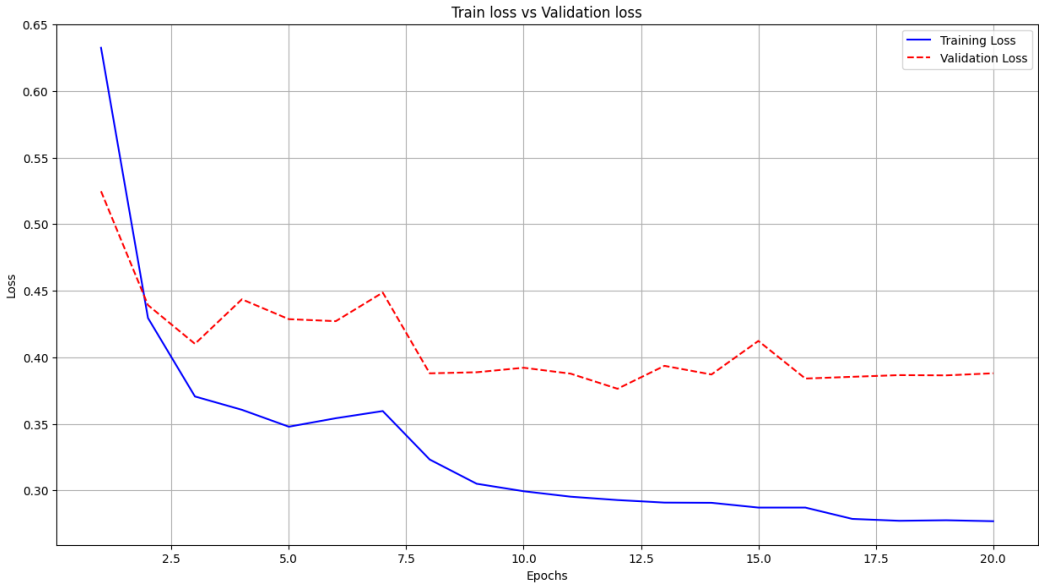


Figure 4: Faster R-CNN Dropout + Regularization Learning Curves



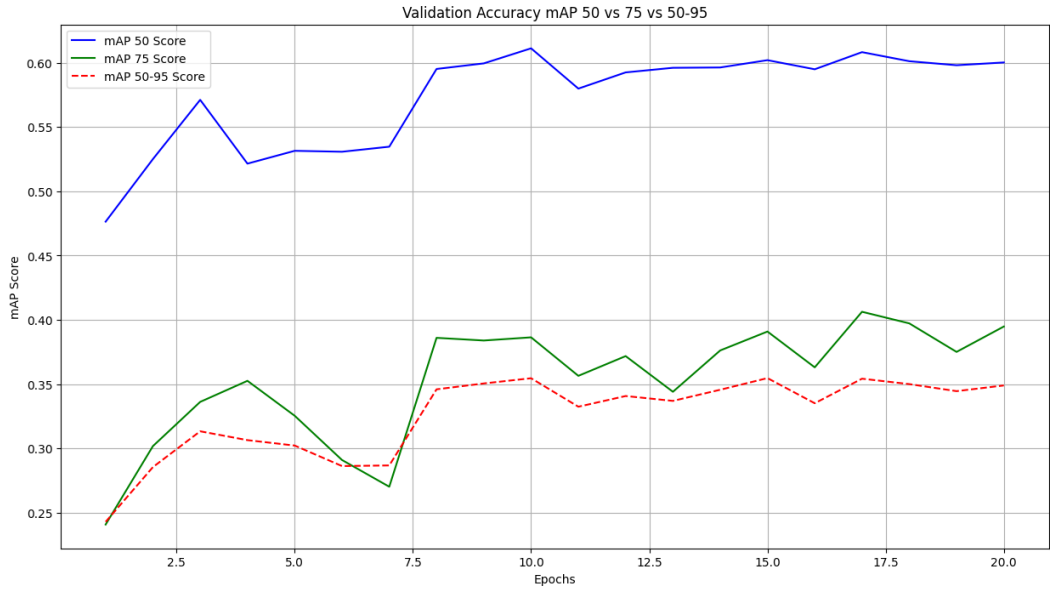


Figure 5: YOLOv5 Learning Curves

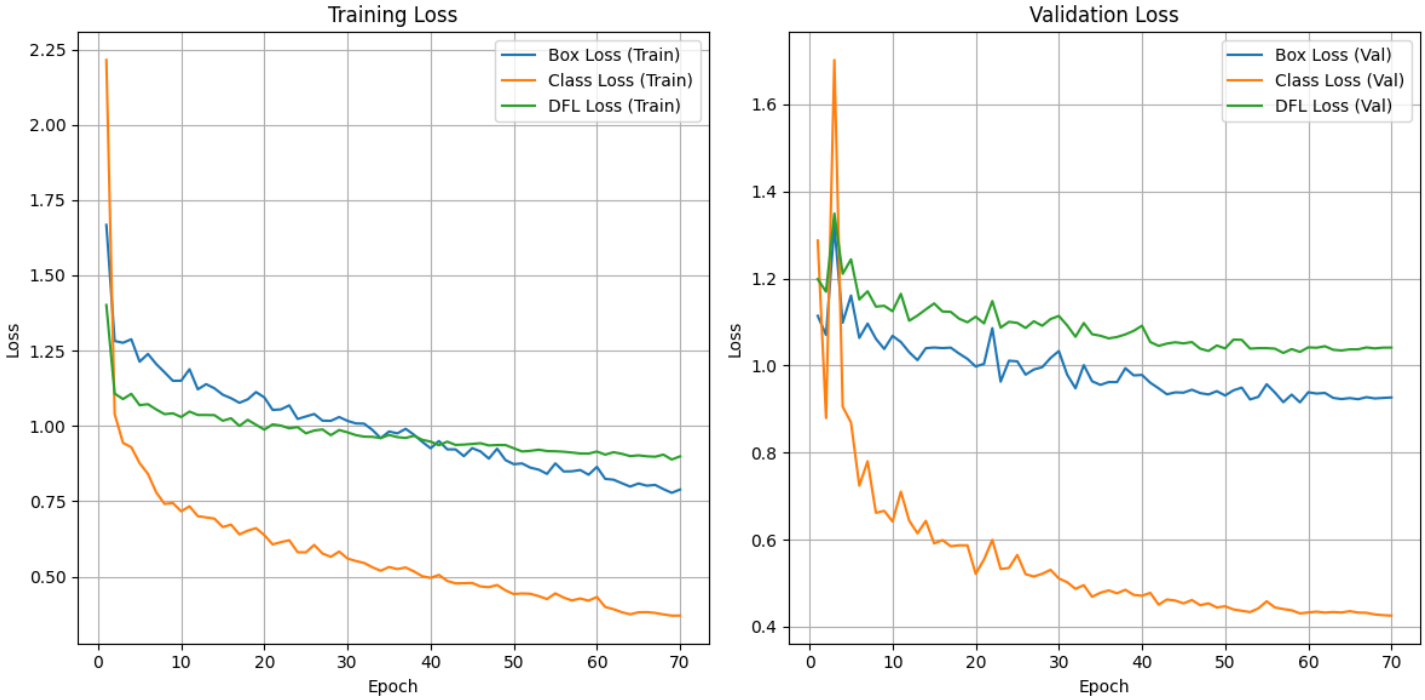


Table 1: Test Scores for Each Architecture

Architecture	**Precision**	**Recall**	**mAP@0.5**	**mAP@0.5-0.95**	**Test Loss**
YOLOv5	0.849	0.922	0.949	0.668	/
Faster R-CNN (default)	0.539	0.574	0.764	0.476	0.341
Faster R-CNN (freeze)	0.529	0.573	0.749	0.475	0.347
Faster R-CNN (dropout)	0.490	0.596	0.688	0.410	0.381
Faster R-CNN (dropout + regularization)	0.462	0.580	0.628	0.385	0.383

Table 2: Class-wise Model Performance

mAP@0.5:0.95 (Per Class)	YOLOv5	Faster R-CNN (default)	Faster R-CNN (freeze)	Faster R-CNN (dropout)	Faster R-CNN (dropout + regularization)
with_mask	0.724	0.609	0.610	0.587	0.583
without_mask	0.627	0.455	0.454	0.417	0.440
mask_worn_incorrectly	0.654	0.364	0.362	0.226	0.132

6 Analysis of Model Performance

The performance comparison between YOLOv5 and Faster R-CNN architectures reveals that YOLOv5 outperforms Faster R-CNN across all metrics. YOLOv5 achieves the highest precision (0.849), recall (0.922), mAP@0.5 (0.949), and mAP@0.5-0.95 (0.668), indicating its superior ability to detect objects accurately and reliably in various scenarios. In contrast, the best-performing Faster R-CNN variant (default) achieves significantly lower metrics, with an mAP@0.5 of 0.764 and mAP@0.5-0.95 of 0.476, demonstrating its relative inefficiency.

A closer look at class-wise performance provides further insights. YOLOv5 achieves a notable mAP@0.5-0.95 for the 'with mask' (0.724), 'without mask' (0.627), and 'mask worn incorrectly' (0.654) classes, outperforming Faster R-CNN in all three categories. In particular, YOLOv5 excels in detecting the minority class, 'mask worn incorrectly', where the best-performing Faster R-CNN variant achieves only 0.364 mAP@0.5-0.95. This highlights YOLOv5's robustness in handling imbalanced datasets and its ability to generalize across varying mask-wearing scenarios.

An interesting observation emerged when analyzing the class-wise results of Faster R-CNN variants. Contrary to expectations, the addition of dropout and regularization, techniques typically used to mitigate overfitting, resulted in a decline in performance, especially for minority classes like 'mask worn incorrectly' (mAP@0.5-0.95 dropped to 0.226 in the dropout variant and 0.132 in the combined variant). This suggests that such regularization techniques may not be well-suited for the Faster R-CNN architecture in this specific task, potentially due to the model's sensitivity to small object features or the limited dataset size.
These results collectively emphasize YOLOv5 as the better choice for real-time face mask detection, especially in scenarios requiring high accuracy for all classes, including the challenging "mask worn incorrectly" category.

7 Conclusions

From the results we obtained, we can tell YOLOv5 is a better fit for real-time scenarios due to its faster speed, efficiency on smaller datasets, and robust performance in detecting small and dense items, which align with the characteristics of our image data and objects.

As for Faster R-CNN, we consider it acceptable as an alternative, however, it faces severe issues of overfitting and also struggles with the accuracy in minority classes, which indicates the current strategies including regularization, dropout, and freezing trainable layers in the backbone have few effects. Thus, we will go with YOLOv5 as the final chosen model for face mask detection.

There are also a couple more strategies to be carried out in future work:

- Upgrade the model version for YOLO to experiment if more sophisticated model (v8) can improve further.
- Optimal the dataset by introducing more high-quality and balanced images ('mask worn incorrectly') to address the class imbalance issue from the origin source.
- Enhance the Faster R-CNN by focusing on its backbone architecture: replacing ResNet50 by lightweight networks such as ResNet18, EfficientNet, RetinaNet, VGG, etc

References

Jocher, G. (2020). *YOLOv5*. Retrieved from https://github.com/ultralytics/yolov5

Kaggle. (2020). *Face Mask Detection dataset*. Retrieved from https://www.kaggle.com/datasets/andrewmvd/face-mask-detection

Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN: Towards real-time object detection with region proposal networks*. Advances in Neural Information Processing Systems, 28, 91–99.